# Metodologie di Programmazione (M-Z)

II semestre - a.a. 2022 – 2023

Testing**

a cura di Mattia Samory*




SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

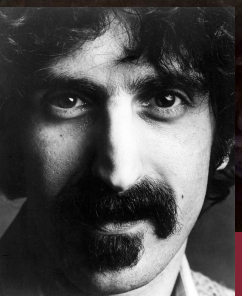
**I crediti sulle slide di questo corso sono riportati nell'ultima slide

# Testing

# 🎯Obiettivi

- Introduzione ad XP
- Testing: cos’è?
- Caratteristiche e tipologie dei test
- Unit Test
- I primi esempi
- Strumenti di lavoro





# XP

Nell'ingegneria del software, per metodologia agile [...] si intende un particolare metodo per lo sviluppo del software che coinvolge quanto più possibile il committente, ottenendo in tal modo una elevata reattività alle sue richieste.

L'Extreme Programming è una metodologia agile e un approccio all'ingegneria del software formulato da Kent Beck,Ward Cunningham e Ron Jeffries.

# Valori XP

- Comunicazione
  - standard per il codice, codice collettivo,  condivisione della conoscenza
- Semplicità
  - principio del “simple is better” o “KISS”,  modifiche incrementali
- Feedback
  - dal sistema, dal cliente, dal team
- Coraggio
  - si dà in uso il sistema il prima possibile e si implementano i cambiamenti richiesti man mano

# Fasi

- 4 fasi
  - progettazione, design, sviluppo e testing
- Per ognuna, cicli di sviluppo brevi e ripetuti
  - pianifica ~ realizza ~ verifica ~ rilascia ~ ...
  - testing ~ sviluppo ~ refactoring ~ …
- Miglioramenti incrementali

# 🎯Obiettivi

- ✓ Introduzione ad XP
- Testing: cos’è?
- Caratteristiche e tipologie dei test
- Unit Test
- I primi esempi
- Strumenti di lavoro

# Testing

è un procedimento utilizzato per individuare le carenze di correttezza, completezza e affidabilità delle componenti software in corso di sviluppo. Consiste nell'eseguire il software da collaudare, da solo o in combinazione ad altro software di servizio, e nel valutare se il comportamento del software rispetta i requisiti.

**debugging**

- aiuta a trovare gli errori (non necessariamente alla fonte)
- si fa dopo la scrittura del codice
- più economico per progetti piccoli

**testing**

- evita l'introduzione di errori
- aumenta la probabilità di correttezza
- permette il rilevamento di errori

# Obiettivi del testing

- Aumentare la qualità
  - il software è corretto? (verifica)
  - stiamo facendo il software che ci siamo prefissi? (validazione)
- Diminuzione del rischio
- Test come specifica
- Test come documentazione (comprensione)
- Aumento della fiducia (facilitazione nel refactoring)

# Miti del testing

“Richiede tempo ed io sono sempre in  ritardo” >:o

- sei in ritardo -> scrivi meno test
- scrivi meno test -> il tuo codice è più fragile
- il tuo codice è fragile -> sei meno produttivo
- sei meno produttivo -> sei più in ritardo prima
- sei più in ritardo di prima -> scrivi ancora meno test
- …

# 🎯Obiettivi

- ✓ Introduzione ad XP
- ✓ Testing: cos’è?
  - Caratteristiche e tipologie dei test
  - Unit Test
  - I primi esempi
  - Strumenti di lavoro

# Caratteristiche

- Ripetibili
- Semplici
- Espressivi
- Robusti

# Caratteristiche

- Non devono introdurre errori
- Non devono prevedere modifiche al codice
- Devono coprire “tutto” il codice
- Facili da eseguire, completamente  automatici

# tipologie

- Customer test
- Unit test
- Integration/Interaction test
- Performance test

- Customer test
  - chiariscono gli obiettivi
  - controllano le funzionalità complessive
- Performance test
- Integration/Interaction test
  - controllano le interazioni tra le parti del  software
- Unit test
  - controllano la correttezza del codice

# 🎯Obiettivi

- ✓ Introduzione ad XP
- ✓ Testing: cos’è?
- ✓ Caratteristiche e tipologie dei test
  - Unit Test
  - I primi esempi
  - Strumenti di lavoro

# unit test

è una procedura usata per controllare che ogni singola unità del codice sorgente si comporti correttamente. Un'unità è la più piccola parte collaudabile di un applicazione

- Un unità in OOP = un metodo di una classe
- Test devono essere indipendenti tra loro
- Ogni unità va testata in isolamento

# refactoring

è un processo di cambiamento del codice di un programma con lo scopo di renderlo **aperto al cambiamento, migliorarne** la leggibilità o **semplificarne** la struttura, mantenendone però **inalterate le funzionalità**.

# Unit test + refactoring

- Non devono introdurre errori
- “small change, test, small change, test, …”
- Qualità totale!

Filosofia d’uso per XP

- Non si deve scrivere una linea di codice senza che sia scritto un test che ne verifichi il corretto funzionamento

# 🎯Obiettivi

- ✓ Introduzione ad XP
- ✓ Testing: cos’è?
- ✓ Caratteristiche e tipologie dei test
- ✓ Unit Test
  - I primi esempi
  - Strumenti di lavoro

# esempio 1

```
public class Hello {
   public static void main (String[]
   argv) {
      System.out.println(”Hello world!”);
   }
}
```

→ non testabile

# esempio 1: presentazione

```
public class Hello {
   public static void main (String[]
   argv) {
      System.out.println(
         HelloLibrary.sayHello(”en”));
   }
}
```

# esempio 1: codice/dati

```
public String sayHello(String language)
throws NotImplementedLanguageException{

   if (language.equals(”en”))
      return ”Hello World!”;
   else
      throw new
   NotImplementedLanguageException();
}
```

# test 1

```
public void helloEnglishTest() {
   String expected = ”Hello World!”;
   String result =
   HelloLibrary.sayHello(”en”);

   assertEquals(”English translation is
   wrong.”,  expected, result);
}
```

volete testare un metodo che calcola il prodotto di due numeri. Quali input testereste?

# quali input?

- Provate tutti gli input è impossibile
- Scegliere quelli più rappresentativi

- Per esempio, con un metodo che calcola il prodotto
  - 5, 5
  - 0, 5 e viceversa
  - 0, -5 e viceversa
  - -5, -5
  - 0, 0

# 🎯Obiettivi

- ✓ Introduzione ad XP
- ✓ Testing: cos’è?
- ✓ Caratteristiche e tipologie dei test
- ✓ Unit Test
- ✓ I primi esempi
- ○ Strumenti di lavoro

# JUnit

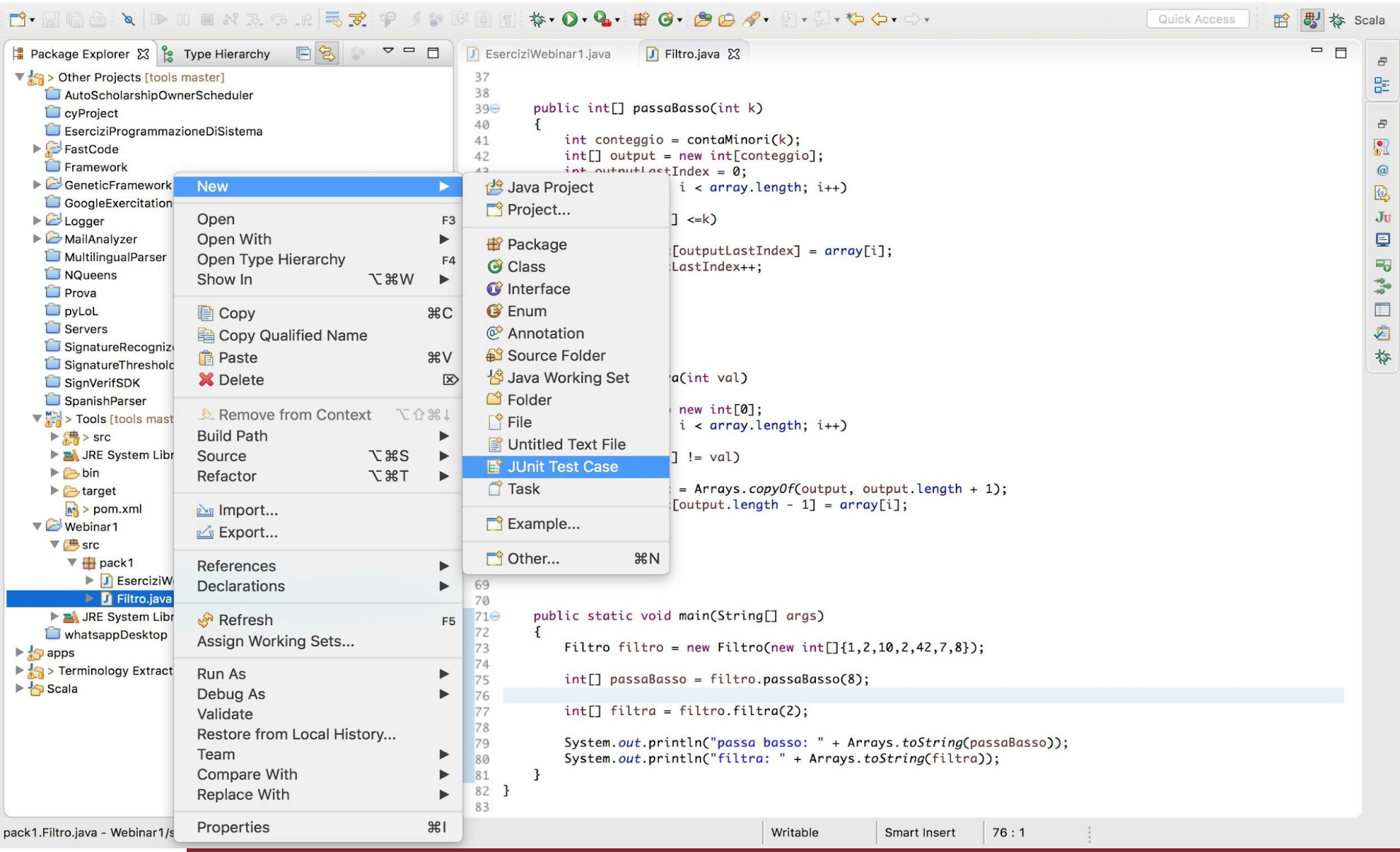
L'obiettivo di JUnit è quello di fornire al programmatore un ambiente che permette di sapere in ogni momento e con poco sforzo se il proprio programma ha superato tutti i test

- Framework per scrivere test scritto da Erich Gamma
- Framework:
  - codice nato per facilitare lo sviluppo di applicazioni
  - tipicamente fornisce alcuni strumenti eseguibili come applicazioni grafiche per eseguire i test
  - stabilisce delle regole di scrittura del codice per poter eseguire questi strumenti

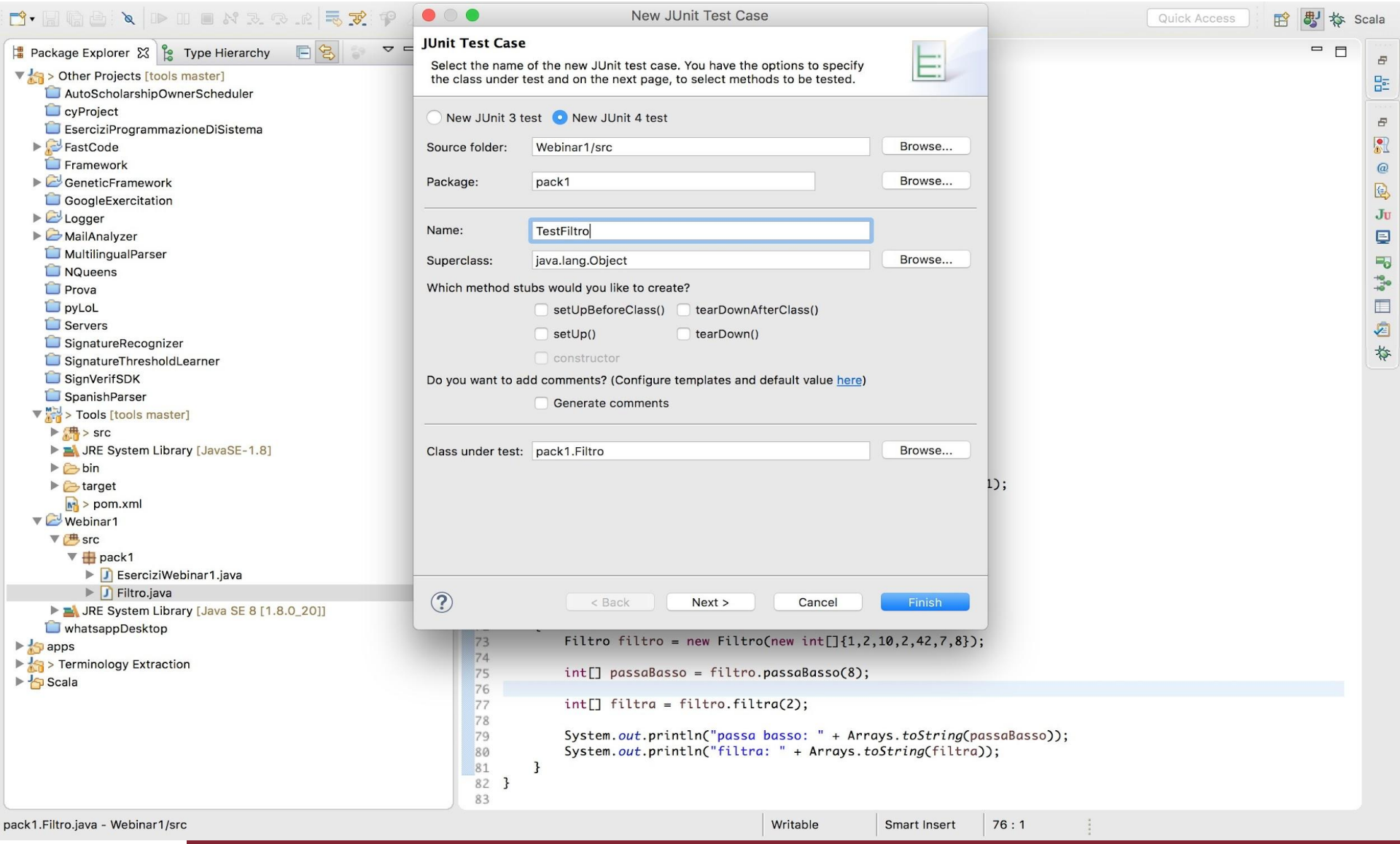
# Junit - Framework per fare unit test in Java.

- Utile per testare singole parti del software (ad esempio un metodo o una classe).
- Integrato in Eclipse.
- Utilizza le annotazioni:
  - **@Test** per i metodi che definiscono un test.
  - **@Before** per i metodi che devono essere eseguiti prima di ogni unit test (utile per l'inizializzazione delle variabili)
  - **@After** per i metodi che devono essere eseguiti dopo ogni unit test (utile per eliminare file creati durante il test)
  - **@BeforeClass @AfterClass** per i metodi che devono essere chiamati una sola volta prima e dopo tutti i test.

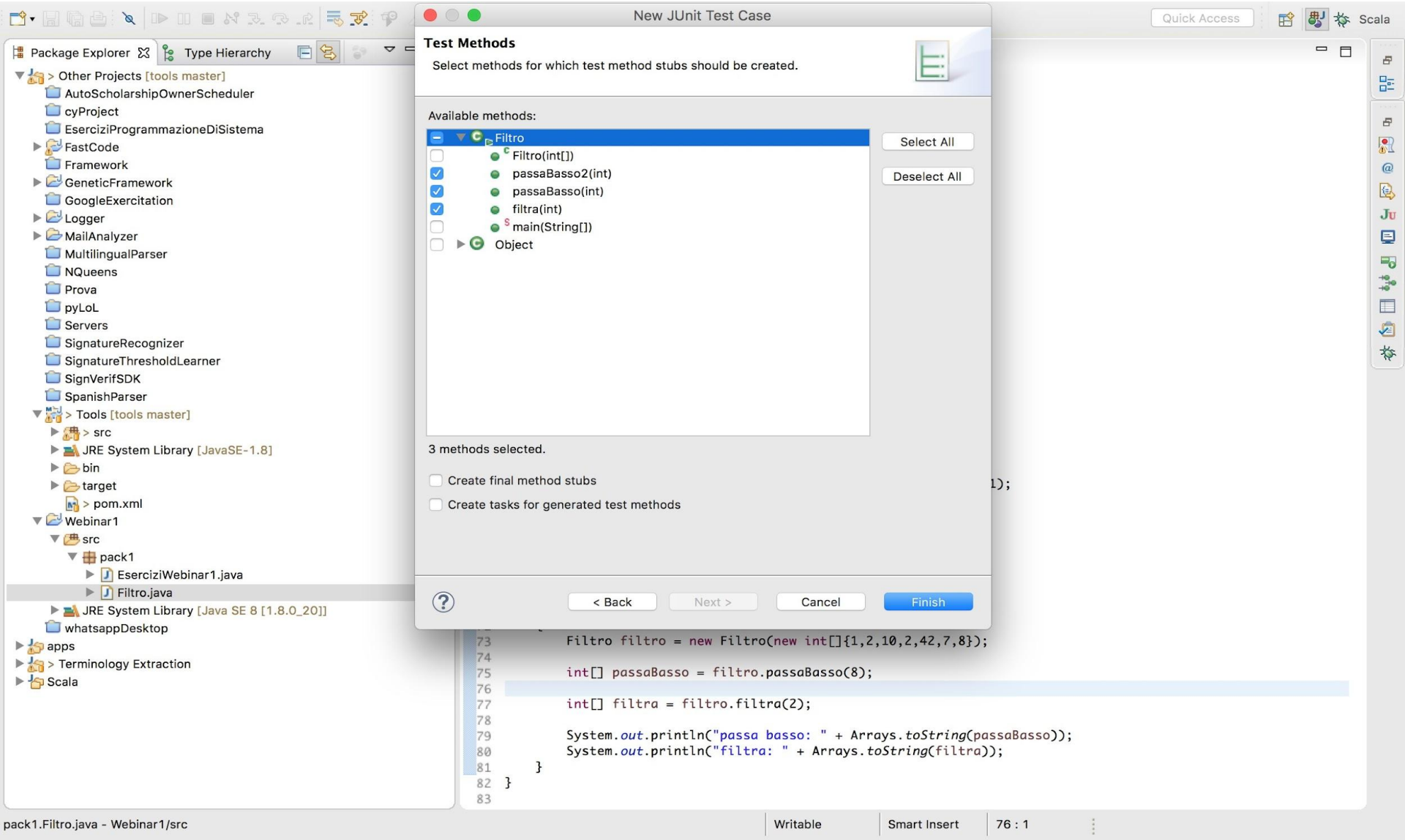
# Junit


Package Explorer
Type Hierarchy
> Other Projects [tools master]
AutoScholarshipOwnerScheduler
cyProject
EserciziProgrammazioneDiSistema
FastCode
Framework
GeneticFramework
GoogleExercitation
Logger
MailAnalyzer
MultilingualParser
NQueens
Prova
pyLoL
Servers
SignatureRecogniz
SignatureThreshold
SignVerifSDK
SpanishParser
> Tools [tools mast
> src
JRE System Libr
bin
target
> pom.xml
Webinar1
src
pack1
EserciziW
Filtro.java
JRE System Libr
whatsappDesktop
apps
> Terminology Extract
Scala
New
Open F3
Open With
Open Type Hierarchy F4
Show In ⌥⌘W
Copy ⌘C
Copy Qualified Name
Paste ⌘V
Delete
Remove from Context ⌥⇧⌘↓
Build Path
Source ⌥⌘S
Refactor ⌥⌘T
Import...
Export...
References
Declarations
Refresh F5
Assign Working Sets...
Run As
Debug As
Validate
Restore from Local History...
Team
Compare With
Replace With
Properties ⌘I
Java Project
Project...
Package
Class
Interface
Enum
Annotation
Source Folder
Java Working Set
Folder
File
Untitled Text File
JUnit Test Case
Task
Example...
Other... ⌘N
EserciziWebinar1.java
Filtro.java
Quick Access
Scala
public int[] passaBasso(int k)
{
int conteggio = contaMinori(k);
int[] output = new int[conteggio];
public static void main(String[] args)
{
Filtro filtro = new Filtro(new int[]{1,2,10,2,42,7,8});
int[] passaBasso = filtro.passaBasso(8);
int[] filtra = filtro.filtra(2);
System.out.println("passa basso: " + Arrays.toString(passaBasso));
System.out.println("filtra: " + Arrays.toString(filtra));
}
}
Writable
Smart Insert
76 : 1

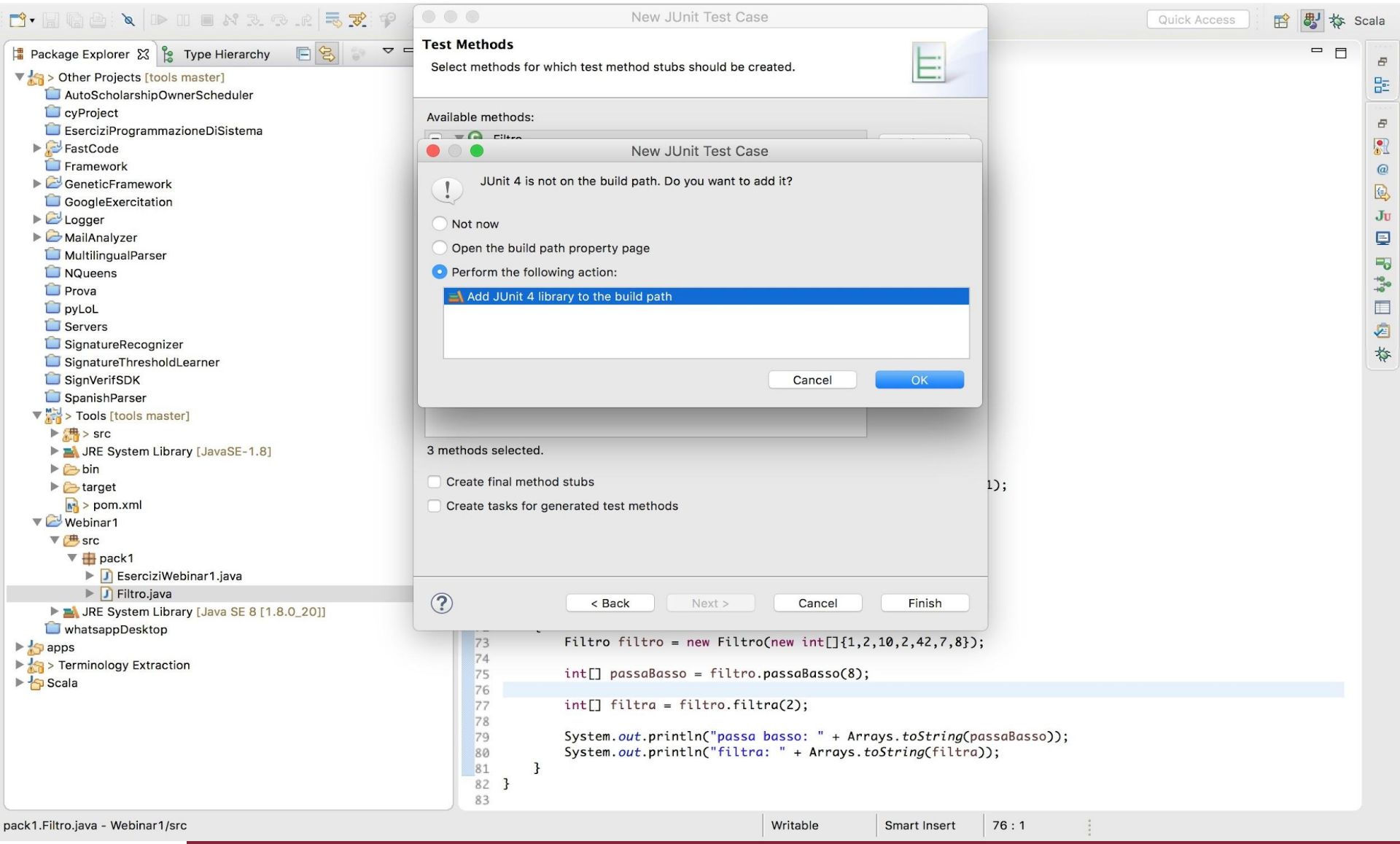
# Junit


New JUnit Test Case
JUnit Test Case
Select the name of the new JUnit test case. You have the options to specify the class under test and on the next page, to select methods to be tested.
New JUnit 3 test
New JUnit 4 test
Source folder: Webinar1/src
Package: pack1
Name: TestFiltro
Superclass: java.lang.Object
Which method stubs would you like to create?
setUpBeforeClass()
tearDownAfterClass()
setUp()
tearDown()
constructor
Do you want to add comments? (Configure templates and default value here)
Generate comments
Class under test: pack1.Filtro
Browse...
< Back
Next >
Cancel
Finish
Package Explorer
Type Hierarchy
Other Projects [tools master]
AutoScholarshipOwnerScheduler
cyProject
EserciziProgrammazioneDiSistema
FastCode
Framework
GeneticFramework
GoogleExercitation
Logger
MailAnalyzer
MultilingualParser
NQueens
Prova
pyLoL
Servers
SignatureRecognizer
SignatureThresholdLearner
SignVerifSDK
SpanishParser
Tools [tools master]
src
JRE System Library [JavaSE-1.8]
bin
target
pom.xml
Webinar1
src
pack1
EserciziWebinar1.java
Filtro.java
JRE System Library [Java SE 8 [1.8.0_20]]
whatsappDesktop
apps
Terminology Extraction
Scala
Filtro filtro = new Filtro(new int[]{1,2,10,2,42,7,8});
int[] passaBasso = filtro.passaBasso(8);
int[] filtra = filtro.filtra(2);
System.out.println("passa basso: " + Arrays.toString(passaBasso));
System.out.println("filtra: " + Arrays.toString(filtra));
}
}
pack1.Filtro.java - Webinar1/src
Writable
Smart Insert
76 : 1

# Junit


New JUnit Test Case
Test Methods
Select methods for which test method stubs should be created.
Available methods:
Filtro
Filtro(int[])
passaBasso2(int)
passaBasso(int)
filtra(int)
main(String[])
Object
Select All
Deselect All
3 methods selected.
Create final method stubs
Create tasks for generated test methods
< Back
Next >
Cancel
Finish
Package Explorer
Type Hierarchy
Other Projects [tools master]
AutoScholarshipOwnerScheduler
cyProject
EserciziProgrammazioneDiSistema
FastCode
Framework
GeneticFramework
GoogleExercitation
Logger
MailAnalyzer
MultilingualParser
NQueens
Prova
pyLoL
Servers
SignatureRecognizer
SignatureThresholdLearner
SignVerifSDK
SpanishParser
Tools [tools master]
src
JRE System Library [JavaSE-1.8]
bin
target
pom.xml
Webinar1
src
pack1
EserciziWebinar1.java
Filtro.java
JRE System Library [Java SE 8 [1.8.0_20]]
whatsappDesktop
apps
Terminology Extraction
Scala
Filtro filtro = new Filtro(new int[]{1,2,10,2,42,7,8});
int[] passaBasso = filtro.passaBasso(8);
int[] filtra = filtro.filtra(2);
System.out.println("passa basso: " + Arrays.toString(passaBasso));
System.out.println("filtra: " + Arrays.toString(filtra));
pack1.Filtro.java - Webinar1/src
Writable
Smart Insert
76 : 1

# Junit


New JUnit Test Case
Test Methods
Select methods for which test method stubs should be created.
Available methods:
New JUnit Test Case
JUnit 4 is not on the build path. Do you want to add it?
Not now
Open the build path property page
Perform the following action:
Add JUnit 4 library to the build path
Cancel
OK
3 methods selected.
Create final method stubs
Create tasks for generated test methods
< Back
Next >
Cancel
Finish
Package Explorer
Type Hierarchy
Other Projects [tools master]
AutoScholarshipOwnerScheduler
cyProject
EserciziProgrammazioneDiSistema
FastCode
Framework
GeneticFramework
GoogleExercitation
Logger
MailAnalyzer
MultilingualParser
NQueens
Prova
pyLoL
Servers
SignatureRecognizer
SignatureThresholdLearner
SignVerifSDK
SpanishParser
Tools [tools master]
src
JRE System Library [JavaSE-1.8]
bin
target
pom.xml
Webinar1
src
pack1
EserciziWebinar1.java
Filtro.java
JRE System Library [Java SE 8 [1.8.0_20]]
whatsappDesktop
apps
Terminology Extraction
Scala
Filtro filtro = new Filtro(new int[]{1,2,10,2,42,7,8});
int[] passaBasso = filtro.passaBasso(8);
int[] filtra = filtro.filtra(2);
System.out.println("passa basso: " + Arrays.toString(passaBasso));
System.out.println("filtra: " + Arrays.toString(filtra));
pack1.Filtro.java - Webinar1/src
Writable
Smart Insert
76 : 1

# Junit

```
package pack1;

import static org.junit.Assert.*;

import org.junit.Test;

public class TestFiltro
{

    @Test
    public void testPassaBasso2()
    {
        fail("Not yet implemented");
    }

    @Test
    public void testPassaBasso()
    {
        fail("Not yet implemented");
    }

    @Test
    public void testFiltra()
    {
        fail("Not yet implemented");
    }

}
```

# Junit

**Ogni test deve testare una singola unità della classe che stiamo controllando!!!**

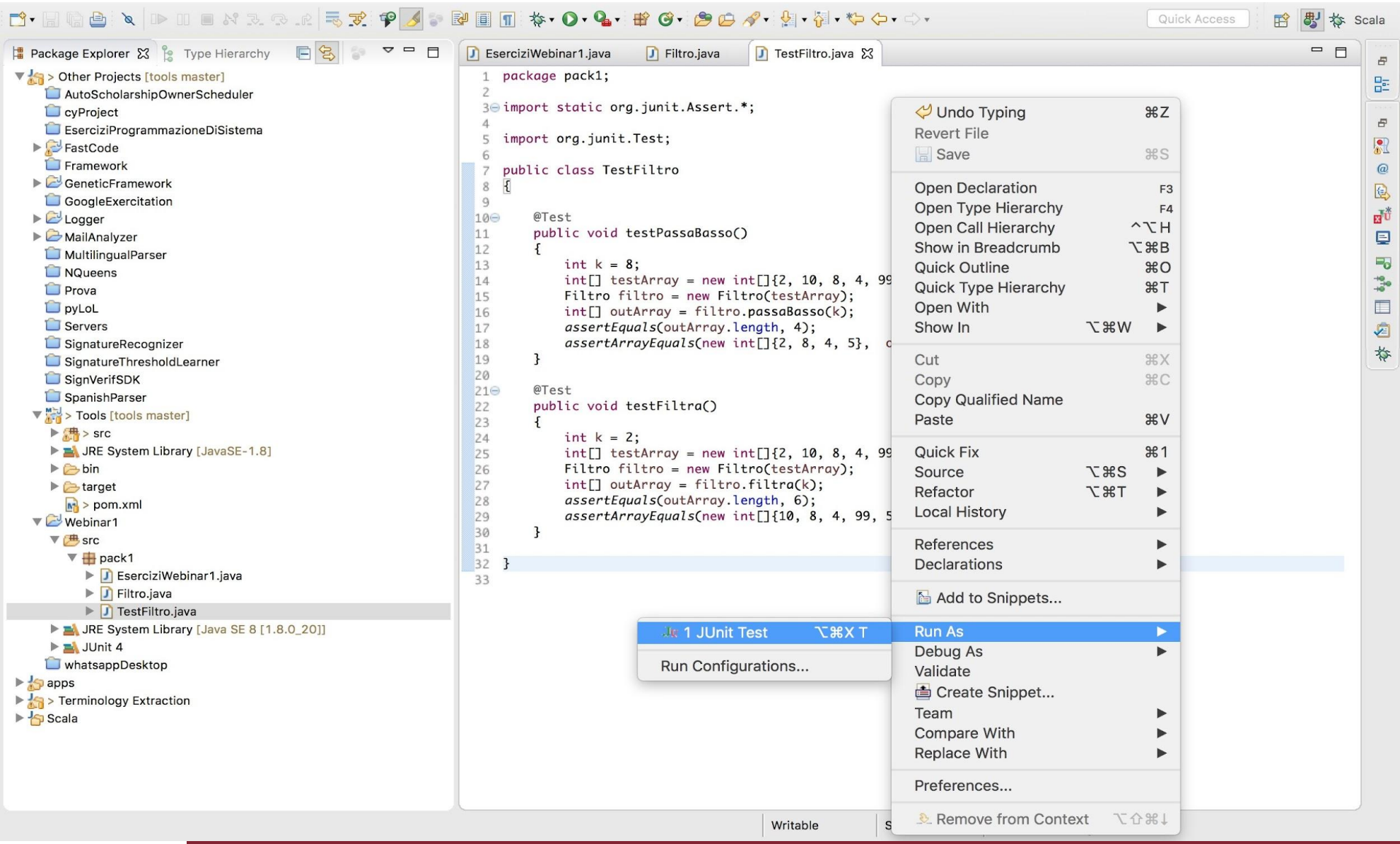
```
@Test
public void testPassaBasso()
{
    int k = 8;
    int[] testArray = new int[]{2, 10, 8, 4, 99, 5, 42};
    Filtro filtro = new Filtro(testArray);
    int[] outArray = filtro.passaBasso(k);
    assertEquals(outArray.length, 4);
    assertArrayEquals(new int[]{2, 8, 4, 5},  outArray);
}
```

# Junit

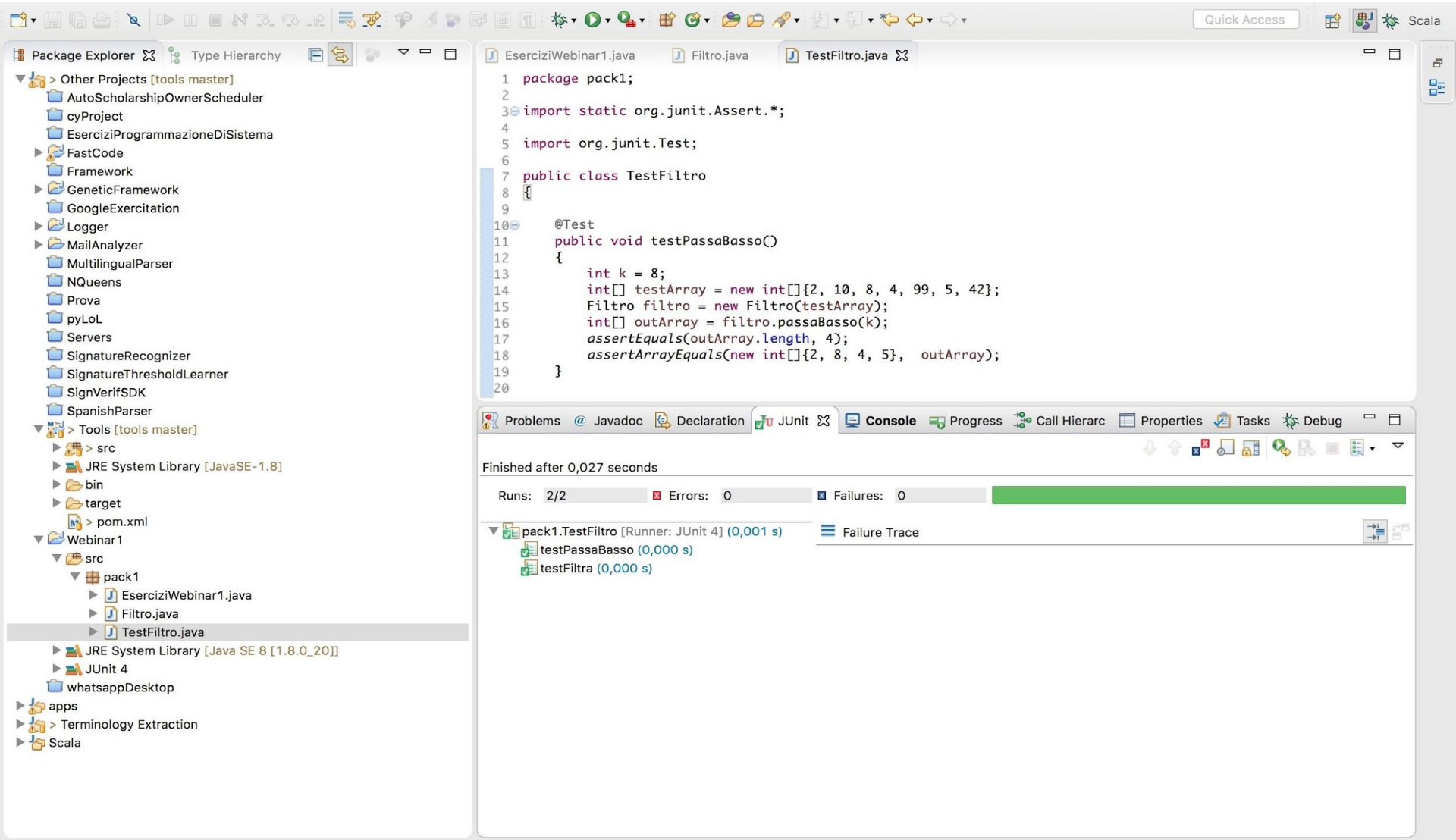
- Nei test vanno utilizzate le **assertion** per verificare che una certa condizione sia vera (o falsa).

| method name / parameters | description |
|---|---|
| `assertTrue(`***test***`)`<br>`assertTrue("`***message***`", `***test***`)` | Causes this test method to fail if the given `boolean` test is not `true`. |
| `assertFalse(`***test***`)`<br>`assertFalse("`***message***`", `***test***`)` | Causes this test method to fail if the given `boolean` test is not `false`. |
| `assertEquals(`***expectedValue***`, `***value***`)`<br>`assertEquals("`***message***`", `***expectedValue***`, `***value***`)` | Causes this test method to fail if the given two values are not equal to each other. (For objects, it uses the `equals` method to compare them.) The first of the two values is considered to be the result that you expect; the second is the actual result produced by the class under test. |
| `assertNotEquals(`***value1***`, `***value2***`)`<br>`assertNotEquals("`***message***`", `***value1***`, `***value2***`)` | Causes this test method to fail if the given two values *are* equal to each other. (For objects, it uses the `equals` method to compare them.) |
| `assertNull(`***value***`)`<br>`assertNull("`***message***`", `***value***`)` | Causes this test method to fail if the given value is not `null`. |
| `assertNotNull(`***value***`)`<br>`assertNotNull("`***message***`", `***value***`)` | Causes this test method to fail if the given value *is* `null`. |
| `assertSame(`***expectedValue***`, `***value***`)`<br>`assertSame("`***message***`", `***expectedValue***`, `***value***`)`<br>`assertNotSame(`***value1***`, `***value2***`)`<br>`assertNotSame("`***message***`", `***value1***`, `***value2***`)` | Identical to `assertEquals` and `assertNotEquals` respectively, except that for objects, it uses the `==` operator rather than the `equals` method to compare them. (The difference is that two objects that have the same state might be `equals` to each other, but not `==` to each other. An object is only `==` to itself.) |
| `fail()`<br>`fail("`***message***`")` | Causes this test method to fail. |

# Junit


Package Explorer
Type Hierarchy
> Other Projects [tools master]
AutoScholarshipOwnerScheduler
cyProject
EserciziProgrammazioneDiSistema
FastCode
Framework
GeneticFramework
GoogleExercitation
Logger
MailAnalyzer
MultilingualParser
NQueens
Prova
pyLoL
Servers
SignatureRecognizer
SignatureThresholdLearner
SignVerifSDK
SpanishParser
> Tools [tools master]
> src
JRE System Library [JavaSE-1.8]
bin
target
> pom.xml
Webinar1
src
pack1
EserciziWebinar1.java
Filtro.java
TestFiltro.java
JRE System Library [Java SE 8 [1.8.0_20]]
JUnit 4
whatsappDesktop
apps
> Terminology Extraction
Scala
EserciziWebinar1.java
Filtro.java
TestFiltro.java
1 package pack1;
3 import static org.junit.Assert.*;
5 import org.junit.Test;
7 public class TestFiltro
8 {
10 @Test
11 public void testPassaBasso()
12 {
13 int k = 8;
14 int[] testArray = new int[]{2, 10, 8, 4, 99
15 Filtro filtro = new Filtro(testArray);
16 int[] outArray = filtro.passaBasso(k);
17 assertEquals(outArray.length, 4);
18 assertArrayEquals(new int[]{2, 8, 4, 5},
19 }
21 @Test
22 public void testFiltra()
23 {
24 int k = 2;
25 int[] testArray = new int[]{2, 10, 8, 4, 99
26 Filtro filtro = new Filtro(testArray);
27 int[] outArray = filtro.filtra(k);
28 assertEquals(outArray.length, 6);
29 assertArrayEquals(new int[]{10, 8, 4, 99, 5
30 }
32 }
1 JUnit Test ⌥⌘X T
Run Configurations...
Undo Typing ⌘Z
Revert File
Save ⌘S
Open Declaration F3
Open Type Hierarchy F4
Open Call Hierarchy ^⌥H
Show in Breadcrumb ⌥⌘B
Quick Outline ⌘O
Quick Type Hierarchy ⌘T
Open With
Show In ⌥⌘W
Cut ⌘X
Copy ⌘C
Copy Qualified Name
Paste ⌘V
Quick Fix ⌘1
Source ⌥⌘S
Refactor ⌥⌘T
Local History
References
Declarations
Add to Snippets...
Run As
Debug As
Validate
Create Snippet...
Team
Compare With
Replace With
Preferences...
Remove from Context ⌥⇧⌘↓
Writable
Quick Access
Scala

# Junit


Package Explorer
Type Hierarchy
> Other Projects [tools master]
AutoScholarshipOwnerScheduler
cyProject
EserciziProgrammazioneDiSistema
FastCode
Framework
GeneticFramework
GoogleExercitation
Logger
MailAnalyzer
MultilingualParser
NQueens
Prova
pyLoL
Servers
SignatureRecognizer
SignatureThresholdLearner
SignVerifSDK
SpanishParser
> Tools [tools master]
> src
JRE System Library [JavaSE-1.8]
bin
target
> pom.xml
Webinar1
src
pack1
EserciziWebinar1.java
Filtro.java
TestFiltro.java
JRE System Library [Java SE 8 [1.8.0_20]]
JUnit 4
whatsappDesktop
apps
> Terminology Extraction
Scala
Quick Access
Scala
EserciziWebinar1.java
Filtro.java
TestFiltro.java
1 package pack1;
2
3 import static org.junit.Assert.*;
4
5 import org.junit.Test;
6
7 public class TestFiltro
8 {
9
10 @Test
11 public void testPassaBasso()
12 {
13 int k = 8;
14 int[] testArray = new int[]{2, 10, 8, 4, 99, 5, 42};
15 Filtro filtro = new Filtro(testArray);
16 int[] outArray = filtro.passaBasso(k);
17 assertEquals(outArray.length, 4);
18 assertArrayEquals(new int[]{2, 8, 4, 5}, outArray);
19 }
20
Problems
Javadoc
Declaration
JUnit
Console
Progress
Call Hierarc
Properties
Tasks
Debug
Finished after 0,027 seconds
Runs: 2/2
Errors: 0
Failures: 0
pack1.TestFiltro [Runner: JUnit 4] (0,001 s)
testPassaBasso (0,000 s)
testFiltra (0,000 s)
Failure Trace

# Junit – Fallimento di un test

- Assicurarsi che il test sia scritto correttamente (chi testa i test?)
- Esultare per aver trovato un possibile bug.
- Correggere l'errore e ripetere il test.

# 🎯Obiettivi

- ✓ Introduzione ad XP
- ✓ Testing: cos’è?
- ✓ Caratteristiche e tipologie dei test
- ✓ Unit Test
- ✓ I primi esempi
- ✓ Strumenti di lavoro

# Credits

Queste slide sono in parte basate su materiale realizzato da Alessandro Calzavara; sono inoltre basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.